

DOM based XSS attacks are done when some client side script processes data from untrusted source, by usually inserting it back into the DOM. For example,

```
var search = document.
getElementById('search').
value;
var results = document.
getElementById('results');
results.innerHTML = 'You
searched for: ' + search;
```

If you can control the input field then you can easily inject a malicious payload and execute it.

XML External Entities (XXE):

Pretty much everyone knows what XML(Extensible Markup Language) is and if you don't, it's a markup language like HTML, but instead of focusing on displaying data it's focused on describing and storing it. It's a fully customised language, which makes all the XML parses in the wild vulnerable by default. So, it's up to you, as the developer, to make sure that the XML parser is not parsing and processing any XML thrown at it. A lot of websites use XML to communicate between the browser and the server, and in doing so almost a standard API or a standard library is used for parsing. As told earlier, parsers have these inherently flawed specifications which make them exploitable by default. Let's go through how XXE actually works. Say the following is an XML specification for a shopping appli-

cation which is used to check the stock amount of a certain product:

```
<?xml version="1.0"
encoding="UTF-8"?>
<stockCheck><productId>381</
productId></stockCheck>
```

Now this XML query doesn't have any protection against XXE attacks, and hence it can be manipulated to exfiltrate data. What we need to do is declare a DOCTYPE(it is used for declaring entities with a certain document type, which can be used later) that will define an entity with a malicious value(in our case, location to some file). After declaring a custom "xxe" DOCTYPE with the SYSTEM value "file:///etc/passwd", the payload will become:

```
<?xml version="1.0"
encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe
SYSTEM "file:///etc/passwd">]>
<stockCheck><productId>&xxe;
</productId></stockCheck>
```

This will make the server spit out the content of '/etc/passwd' (this file is present by default on all the Linux machines, containing a list of users available on the system alongside information related to each of them). There are a lot of ways to exploit XXE vulns, with even some cases of XXE being successfully executed through file uploads!

File Inclusion Attacks: Even though these kind of attacks are not included in OWASP top 10 (OWASP is the leading organisation making efforts to improve

web application security, check them out: <https://owasp.org/>), we thought it might be best to include them here since we have encountered them in a lot of websites. There are mainly two types of file inclusion attacks: Local and Remote. As the name suggests, in LFI(Local File Inclusion) allows the attacker to read files present on the localhost of the website (i.e the web server). This can lead to exposure of source code and other important files. Similarly, RFI (Remote File Inclusion) lets a file served by the attacker executed by the website, thinking as its own. This is more serious, as while LFI only allows READ access while using RFI, the attacker can execute maliciously crafted scripts.

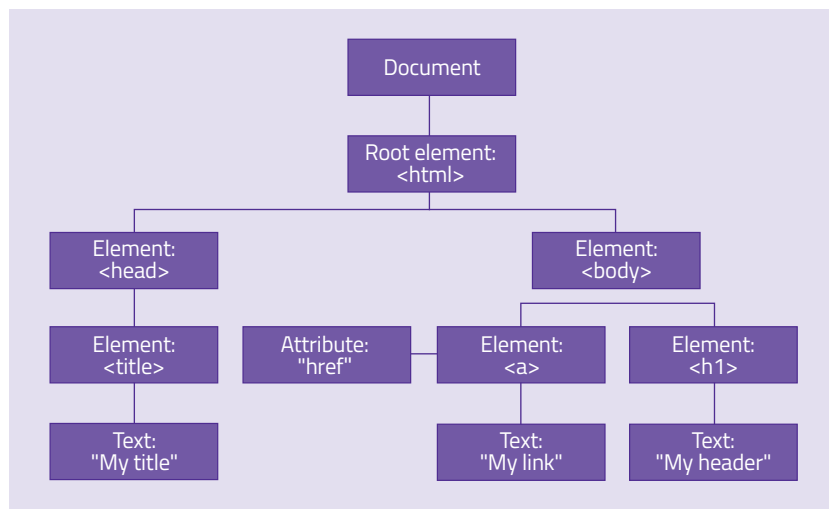
But hackers are one of the most relentless of the human forms. Even though LFI only lets you read the contents of the files, if you mix some genius with it the results can be damaging. Every web server based on Linux has a file 'proc/self/environ' which logs the HTTP requests made to the server. If you have seen an HTTP request before, it contains a 'User-Agent' field which holds the browser metadata. Yes, you guessed it right, user-agents are completely customisable. There, you got write access and you didn't even realise it. For example, using an already existing LFI, you are able to read the '/proc/self/environ' file and it contains something like this:

```
.....HTTP_USER_
AGENT=Opera/9.80 (Windows NT
5.1; U; en) Presto/2.2.15 Ver-
sion/10.00.....
```

The next request you make, using any HTTP request editor tool(BURP Suite is highly recommended), change the 'User-Agent' field to some php code:

```
GET vulnerable.
php?filename=../../../../proc/
self/environ HTTP/1.1
User-Agent: <?php phpinfo();
?>
```

Next time you access this file using LFI, the browser will parse the php code and execute it, essentially giving you remote code execution (RCE). This is one of the techniques that fall under "log poisoning", where creating custom requests



Document Object Structure